Lab 2 Report

Proportional Control



Prepared by:

Junaid Allan Pieters
PTRJUN003

Prepared for:

EEE3094S Control Systems Engineering
Department of Electrical Engineering
University of Cape Town

11 October 2026

Contents

List of Figures iii

List of Tables iv

1 Introduction 1
- 1.1 Overview . . . 1
- 1.2 Aim . . . 1
- 1.3 Objectives . . . 1

2 System Modelling 2
- 2.1 Methodology . . . 2
- 2.2 Modelling . . . 2
  - 2.2.1 Free-Body and Block Diagrams . . . 2
  - 2.2.2 Dynamic Model . . . 2
  - 2.2.3 Closed-Loop Representation . . . 3
  - 2.2.4 Steady-State Properties . . . 3

3 System Identification 4
- 3.1 Introduction . . . 4
- 3.2 Methodology . . . 4
  - 3.2.1 Step-Testing . . . 4
- 3.3 Experimental Setup and Data Files . . . 4
- 3.4 Data Processing . . . 5
  - 3.4.1 Velocity Computation . . . 5
  - 3.4.2 Estimation of Gain and Time Constant . . . 5
  - 3.4.3 Sensor Calibration (V/m) . . . 5
- 3.5 Results and Analysis . . . 6
  - 3.5.1 Step Response Analysis . . . 6
  - 3.5.2 Identification Results . . . 6
- 3.6 Transfer Function Approximation . . . 6
- 3.7 Model Validation . . . 7

4 Controller Design 8
- 4.1 Design Process . . . 8
  - 4.1.1 Plant for Design . . . 8
  - 4.1.2 Design Specifications . . . 8
  - 4.1.3 Gain Selection (Root Locus) . . . 8
  - 4.1.4 Design Summary . . . 9
- 4.2 Simulink Model and Results . . . 10
  - 4.2.1 Block Diagram Implementation . . . 10
  - 4.2.2 Comparison with the Supplied Simulation . . . 10
  - 4.2.3 Controller Hardware Implementation . . . 11

5 Controller Tests 12
- 5.1 Connected Test Setup and Adjustment . . . 12
- 5.2 Experimental Results and Comparison . . . 12

5.3 Performance Analysis . . . 13
5.4 Discussion . . . 13

6 Conclusion and Recommendations **14**
6.1 Conclusion . . . 14
6.2 Recommendations . . . 14
6.3 Final Remarks . . . 14

References **R1**

A Photographs of the Final Controller **A1**

B Experimental Demonstration Record **A2**

C Simulink Configuration and Reproduction **A3**

D Complete Circuit and Interface **A5**
D.1 Signal Stages . . . A5
D.2 Bias Network and DAQ Interface . . . A6
D.3 Bias Loading and Stage Gain Calculations . . . A7

E Detailed Calculations **A8**
E.1 Model Derivation and Signal Definitions . . . A8
E.2 Identification and Timing . . . A9
E.3 Root-Locus Geometry and Gain Bound . . . A10
E.4 Selected Gain and Pole Calculations . . . A11
E.5 Root-Locus Angle and Magnitude Checks . . . A11
E.6 Nominal Step and Settling Calculation . . . A12
E.7 Aerodynamic and Component Uncertainty . . . A13
E.8 Practical Gain and Ratio Correction . . . A14
E.9 Reported Potentiometer Adjustment and Command Calculations . . . A15
E.10 Experimental Metrics and Uncertainty . . . A16

F Additional Validation Plots **A17**

G Identification Data and Code **A20**
G.1 Data Provenance . . . A20
G.2 Organised Source Code . . . A20

H Repository and Reproduction **A21**
H.1 Build the Report . . . A21
H.2 GitHub Folder Links . . . A22
H.3 Page Count and Reproduction . . . A22

List of Figures

2.1 Vertical forces on the normalised Kitticopter. Drag reverses with velocity. . . . 2
2.2 Altitude feedback loop, with voltage scaling, hover bias and command saturation. 3
3.1 Recorded identification input and outputs. The sensor-voltage plateau is clipping at 10 V; altitude continues increasing. Code: Python; MATLAB. Data: results. 4
3.2 Velocity identification. The steady tail determines gain; interpolation and fitting determine the time constant. Code: Python; MATLAB. Data: results. 6
3.3 Sensor calibration from unsaturated pairs and velocity residuals outside the fitting interval. Code: Python; MATLAB. Data: results. 7
4.1 Proportional root locus. The selected poles satisfy the 30% overshoot boundary; the 30 s envelope target is beyond the attainable complex branches. Code: Python; MATLAB. Data: results. 9
4.2 Supplied Simulink architecture. Native model-building and execution scripts: Simulink folder. 10
4.3 Supplied Simulink altitude, replotted and compared with Python for the same reference sequence. Code: Python; MATLAB. Data: results. 10
4.4 Three-stage controller with bias and DAQ connections. Editable source: KiCad folder. 11
5.1 Connected controller response and nominal prediction for the same 5.033 m reference. Code: Python; MATLAB. Data: results. 12
A.1 Supplied final Veroboard photograph. The three op-amp stages and adjustable bias trimmer are visible. A1
B.1 Original controller-demonstration screenshot from the laboratory application, retained as required evidence. The numerical response is replotted in Figure 5.1. A2
C.1 Full altitude and command comparison for the supplied Simulink sequence. Code: Python; MATLAB. Data: results. A4
C.2 Isolated nominal 0-to-5 m design test, showing altitude and command. This is a distinct reference sequence from the supplied scope. Code: Python; MATLAB. Data: results. A4
D.1 Bias potentiometer represented by its two segments, shunt resistor and end resistors. No capacitors are included in the revised KiCad drawing. A6
E.1 Loop-resistor corners. Dashed limits are 30% overshoot and 8% tracking error. The histograms show corner counts, not probabilities. Code: Python; MATLAB. Data: results. A14
F.1 Position-domain model comparison and residuals for the retained identification run. Code: Python; MATLAB. Data: results. A17
F.2 Predicted altitude responses for $b = 0.9b_0, b_0, 1.1b_0$, with local thrust slope held fixed. Code: Python; MATLAB. Data: results. A18
F.3 Illustrative simulated response to +2 mV actuator bias added at 70 s. The shaded region is the 8% tracking band. Code: Python; MATLAB. Data: results. A19

List of Tables

1.1 Laboratory requirements and their evaluation measures. 1
3.1 Identification estimates and adopted design parameters. 6
3.2 Validation errors using the rounded design model. 7
4.1 Manual nominal-controller calculations. 9
4.2 Controller component values and stage calculations. 11
5.1 Model, supplied Simulink and connected-controller comparison. 13
5.2 Evaluation against the laboratory requirements. 13
B.1 Numerical readouts transcribed from the retained test display. A2
C.1 Simulink parameters and test conditions. A3
D.1 DAQ and supply connections. A6
E.1 Model notation and units. A8
E.2 Selected identification samples and backward-difference results. A9
E.3 Independent loop-resistor corner predictions. A13
E.4 Comparison of calculation, simulation and retained experiment. A16
F.1 Predicted performance under aerodynamic uncertainty. A17
G.1 Code modules and their calculation responsibilities. A20
H.1 Clickable evidence map and corresponding Git folders. A21

Chapter 1

Introduction

1.1 Overview

This laboratory designs and tests an analogue proportional controller for the Kitticopter altitude plant. A data-acquisition (DAQ) interface connects the physical controller to simulated vertical motion. Session 1 identifies the plant; Session 2 designs, implements and validates the Veroboard controller [2].

1.2 Aim

The aim is to regulate altitude using proportional control while satisfying the four requirements in the 2026 laboratory brief [1]. Table 1.1 translates these requirements into quantities that can be checked against the measured and simulated responses.

Table 1.1: Laboratory requirements and their evaluation measures.

Requirement	Evaluation measure
Tracking accuracy $> 92\%$	$100(1 - \lvert x - y_\infty \rvert / \lvert x \rvert) > 92\%$ for a nonzero step reference.
Overshoot $< 30\%$	$M_p = 100(y_{\max} - y_\infty)/(y_\infty - y_0) < 30\%$.
Aerodynamic uncertainty $\pm 10\%$	Vary the identified drag coefficient b by $\pm 10\%$, keeping the thrust slope fixed.
Component tolerance $\pm 10\%$	Examine changes in loop gain, reference/feedback ratio and hover bias.

1.3 Objectives

1. Derive the velocity and position models from the free-body diagram and identify the unknown drag, thrust and sensor parameters.
2. Differentiate the recorded altitude, calculate the velocity gain and time constant, calibrate the sensor and validate the resulting model.
3. Select and justify the proportional gain using root-locus and time-response calculations; check the nominal 5 m step and uncertainty cases.
4. Document the three-stage controller circuit and assess the experimental response against the requirements.

The supplied CSV, circuit files and test display support one identification run and one controller test. Numerical plots use Python with MATLAB reproduction code. Main-text Simulink and KiCad diagrams link to their sources; photographs and full derivations are in the appendices.

Chapter 2

System Modelling

2.1 Methodology

A control-oriented model is obtained by applying Newton's second law, linearising the actuator around hover and taking the Laplace transform. The laboratory permits unit mass, constant gravity and drag proportional to vertical velocity [1]. Upward altitude is positive; the unknown parameters are identified in Section 3. Table E.1 defines all symbols and units.

2.2 Modelling

2.2.1 Free-Body and Block Diagrams

Figure 2.1 shows thrust f_t opposing weight g and drag $b\dot{y}$. The complete voltage/altitude architecture is shown in Figure 2.2.

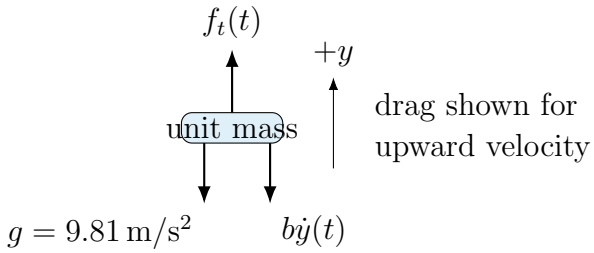


Figure 2.1: Vertical forces on the normalised Kitticopter. Drag reverses with velocity.

2.2.2 Dynamic Model

For mass normalised to one, Newton's second law gives

$$\ddot{y} = f_t - b\dot{y} - g + d_a. \tag{2.1}$$

At hover, $f_{t,h} = g$ and $u_h = 2.5$ V. The local actuator relation is $f_t = g + k_T(u - u_h)$, where the thrust slope k_T is obtained from data. Substitution cancels nominal gravity:

$$\ddot{y} + b\dot{y} = k_T\delta u + d_a, \qquad \delta u = u - u_h. \tag{2.2}$$

The thrust-to-velocity plant is $V(s)/\Delta F_t(s) = 1/(s+b)$. Multiplication by k_T gives the voltage-input model. With zero initial conditions and $d_a = 0$, the voltage-input plants are

$$G_v(s) = \frac{V(s)}{\Delta U(s)} = \frac{k_T}{s+b} = \frac{A}{\tau s + 1}, \quad \tau = \frac{1}{b}, \quad A = \frac{k_T}{b}, \tag{2.3}$$

$$G_y(s) = \frac{Y(s)}{\Delta U(s)} = \frac{G_v(s)}{s} = \frac{A}{s(\tau s + 1)}. \tag{2.4}$$

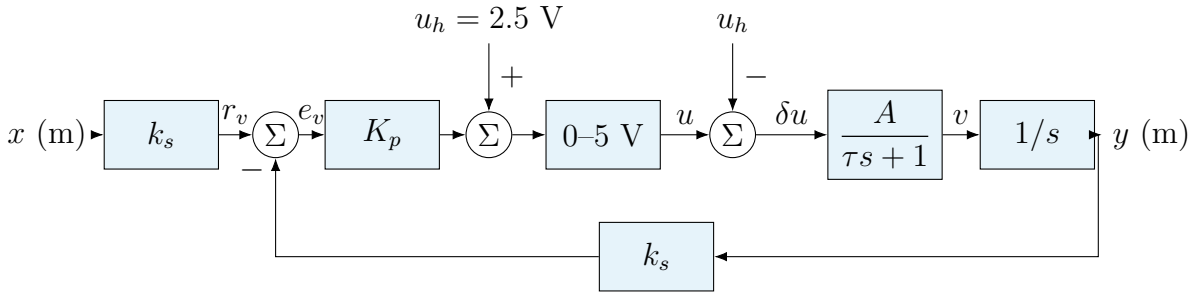


Figure 2.2: Altitude feedback loop, with voltage scaling, hover bias and command saturation.

2.2.3 Closed-Loop Representation

The reference and sensor voltages are $r_v = k_s x$ and $y_v = k_s y$. The command law is

$$u = \mathrm{sat}_{[0,5]}\{2.5 + K_p(r_v - y_v)\}. \tag{2.5}$$

Figure 2.2 applies k_s once to each altitude signal. The hover terms cancel in the deviation plant. The stricter 0–5 V interface range is used for the command; the op-amp supply rails are separate. In the unsaturated region, $\Delta U = K_p k_s (X - Y)$, so

$$L(s) = K_p k_s \frac{A}{s(\tau s + 1)}, \tag{2.6}$$

$$\frac{Y(s)}{X(s)} = \frac{L(s)}{1 + L(s)} = \frac{q}{D(s)}, \quad q = AK_p k_s, \quad D(s) = \tau s^2 + s + q, \tag{2.7}$$

$$\frac{E_y(s)}{X(s)} = \frac{\tau s^2 + s}{D(s)}, \qquad E_y = X - Y. \tag{2.8}$$

An acceleration disturbance enters Equation (2.2) directly:

$$\frac{Y(s)}{D_a(s)} = \frac{\tau}{D(s)}, \qquad \frac{Y(s)}{G(s)} = -\frac{\tau}{D(s)} \quad \text{for uncompensated gravity.} \tag{2.9}$$

2.2.4 Steady-State Properties

For $K_p > 0$, the characteristic polynomial $s^2 + bs + k_T K_p k_s$ is stable. The loop is Type 1, giving zero ideal step-reference error, finite ramp error and infinite parabolic-reference error. A constant command bias is not rejected with zero offset:

$$y_\infty - x_\infty = \frac{d_0}{k_T K_p k_s} = \frac{\delta u_b}{K_p k_s}. \tag{2.10}$$

At 5 m and $K_p = 0.01432$, the 8% error allowance requires $|\delta u_b| < 0.08(5)(0.01432)(0.93) = 5.327$ mV. Full final-value calculations are in Appendix E.

Chapter 3

System Identification

3.1 Introduction

Differentiating altitude removes the position integrator, giving a first-order velocity response from which gain and time constant are estimated. Sensor calibration uses unsaturated voltage/altitude pairs.

3.2 Methodology

3.2.1 Step-Testing

The retained Session 1 command record starts at rest near 2.5 V and changes to 5 V. This voltage-identification run is distinct from the connected three-op-amp controller test described in Section 5. The nominal input increment is $\Delta u = 5.0 - 2.5 = 2.5$ V. The final voltage is held long enough to identify a steady velocity. Figure 3.1 shows the input, position and sensor channels from this run.

3.3 Experimental Setup and Data Files

The Session 1 data folder contains the unmodified `PTRJUN003_raw.csv`; Appendix G gives its provenance. Of 1,634 rows, the first 31 precede a clock reset from 9.5 to 0.1 s and contain zero altitude. They are excluded from analysis. The retained 1,603 samples span 0.1–160.3 s at 0.1 s spacing. The reset is not another trial.

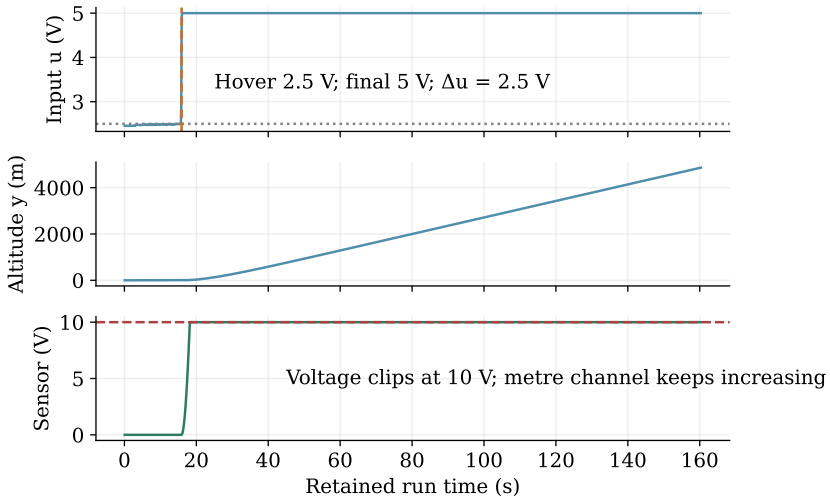


Figure 3.1: Recorded identification input and outputs. The sensor-voltage plateau is clipping at 10 V; altitude continues increasing. Code: Python; MATLAB. Data: results.

3.4 Data Processing

3.4.1 Velocity Computation

The metre channel is differentiated using a backward difference:

$$v_i = \frac{y_i - y_{i-1}}{t_i - t_{i-1}}, \qquad t_{v,i} = \frac{t_i + t_{i-1}}{2}. \tag{3.1}$$

For example, the 16.9 and 17.0 s positions are 2.256 and 2.703 m, giving $v = (2.703 - 2.256)/0.1 = 4.47$ m/s at midpoint 16.95 s. No smoothing is applied. The stored speed column agrees to CSV rounding. Differentiating the clipped voltage channel would incorrectly produce zero velocity; the metre channel avoids this error.

3.4.2 Estimation of Gain and Time Constant

The 404 velocity samples with endpoint time $t \geq 120$ s give $v_\infty = 35.68748$ m/s. With $v_0 = 0$,

$$A = \frac{v_\infty - v_0}{\Delta u} = \frac{35.68748 - 0}{5.0 - 2.5} = 14.27499\,\mathrm{m/(s\,V)}. \tag{3.2}$$

The first-order step response is $v(t) = v_\infty[1 - e^{-(t-t_0)/\tau}]$. At one time constant, $1 - e^{-1} \approx 0.632$, so $v_{63} = 0.632(35.68748) = 22.55448$ m/s. The measured crossing lies between 24.0 and 24.1 s:

$$t_{63} = 24.0 + \frac{22.55448 - 22.53}{22.69 - 22.53}(24.1 - 24.0) = 24.01530\,\mathrm{s}, \tag{3.3}$$

$$\tau_{63} = t_{63} - t_0 \approx 24.01530 - 15.9 = 8.11530\,\mathrm{s}. \tag{3.4}$$

A least-squares exponential fit over 17–60 s gives $V_\infty = 35.68747$ m/s, $\tau = 8.10393$ s and $t_0 = 15.91584$ s. Endpoint-labelled differences are used in that fit; the midpoint shift is absorbed in t_0, leaving τ unchanged.

3.4.3 Sensor Calibration (V/m)

Using 22 nonzero pairs with $0 < y_v < 9.9$ V, the zero-intercept least-squares coefficient is

$$k_s = \frac{\sum_i y_i y_{v,i}}{\sum_i y_i^2} = 0.9300026\,\mathrm{V/m} \approx 0.93\,\mathrm{V/m}. \tag{3.5}$$

The single-point check at 17.0 s is $2.513/2.703 = 0.92971$ V/m. The regression uses multiple unsaturated pairs rather than the clipped 10 V tail. Full processing equations and selected sample calculations are in Appendix E.

3.5 Results and Analysis

3.5.1 Step Response Analysis

Figure 3.2 compares the differentiated measurements with the exponential fit and shows the 63.2% crossing. Velocity tends to a constant while altitude continues rising, as expected for the lag-plus-integrator model.

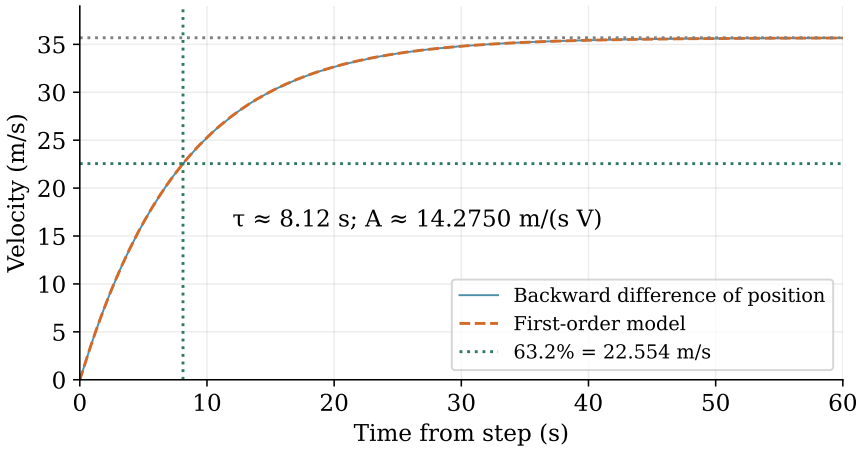


Figure 3.2: Velocity identification. The steady tail determines gain; interpolation and fitting determine the time constant. Code: Python; MATLAB. Data: results.

3.5.2 Identification Results

Table 3.1 compares the two estimation methods. These are estimates from one run, not independent repeated trials. The rounded parameters reproduce the values used in the supplied Simulink model.

Table 3.1: Identification estimates and adopted design parameters.

Method	A [m/(s V)]	τ [s]	k_s [V/m]
Tail / 63.2% crossing	14.27499	8.11530	—
Exponential / sensor fit	14.27499	8.10393	0.930003
Adopted model	14.2748	8.1000	0.9300

3.6 Transfer Function Approximation

Substitution into Equation (2.3) and Equation (2.4) gives

$$\boxed{G_v(s) = \frac{14.2748}{8.1s + 1}, \qquad G_y(s) = \frac{14.2748}{s(8.1s + 1)}}. \tag{3.6}$$

The physical coefficients follow directly: $b = 1/8.1 = 0.1234568$ s^{-1} and $k_T = A/\tau = 14.2748/8.1 = 1.762321$ m/(s^2 V). The differences between fitted and rounded values are small relative to the specified 10% aerodynamic uncertainty.

3.7 Model Validation

The rounded model is evaluated over both the fitting interval (17–60 s) and a held-out tail ($t > 60$ s). It predicts

$$v_{\mathrm{mod}}(t) = 2.5A(1 - e^{-r/\tau}), \tag{3.7}$$

$$y_{\mathrm{mod}}(t) = 2.5A\{r - \tau(1 - e^{-r/\tau})\}, \qquad r = \max(t - t_0, 0). \tag{3.8}$$

The effective onset fitted above is retained. Figure 3.3 checks sensor linearity and velocity residuals; Table 3.2 reports error magnitudes.

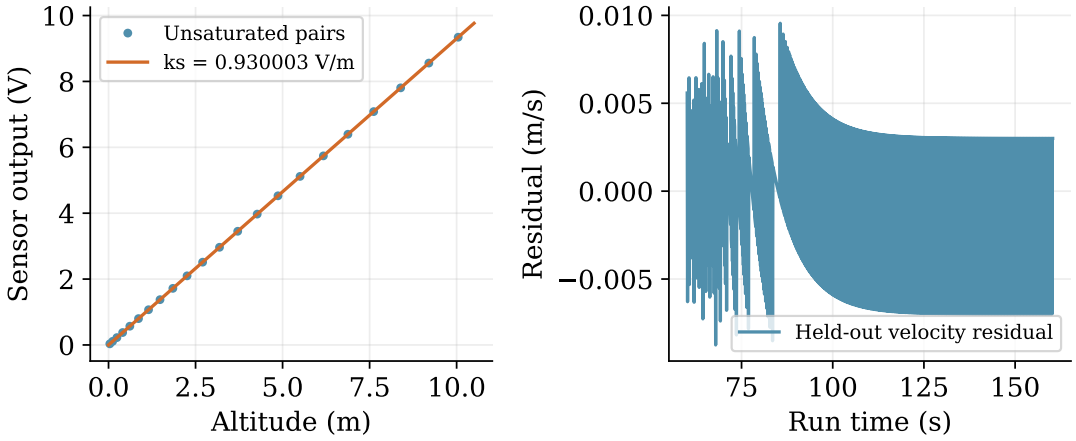


Figure 3.3: Sensor calibration from unsaturated pairs and velocity residuals outside the fitting interval. Code: Python; MATLAB. Data: results.

Table 3.2: Validation errors using the rounded design model.

Comparison	Interval [s]	RMSE
Velocity in fitting window	17–60	0.00533 m/s
Held-out velocity	> 60	0.00436 m/s
Held-out position	> 60	1.686 m

RMSE is $\sqrt{N^{-1}\sum_i (z_i - z_{\mathrm{mod},i})^2}$. Relative to final velocity, the held-out error is 0.0122%. The position comparison in Figure F.1 reaches almost 4,865 m; small rounding and timing differences accumulate through integration, so its absolute RMSE is larger.

The residuals support the first-order velocity structure and the adopted parameter values for controller design. This validation tests another interval of the same run. It does not establish repeatability across independent experiments or the accuracy of a model-check submission to the laboratory system.

Chapter 4

Controller Design

4.1 Design Process

4.1.1 Plant for Design

The design uses Equation (3.6), $k_s = 0.93$ V/m and a 2.5 V hover bias. Its characteristic equation is

$$1 + K_p k_s G_y(s) = 0 \quad \Rightarrow \quad 8.1s^2 + s + 14.2748(0.93)K_p = 0. \tag{4.1}$$

4.1.2 Design Specifications

For a standard underdamped second-order response,

$$M_p = 100e^{-\pi\zeta/\sqrt{1-\zeta^2}}, \quad M_p < 30\% \Rightarrow \zeta > \frac{-\ln 0.3}{\sqrt{\pi^2 + \ln^2 0.3}} = 0.357857. \tag{4.2}$$

Matching the monic form of Equation (4.1) to $s^2 + 2\zeta\omega_n s + \omega_n^2$ yields

$$\omega_n = \sqrt{\frac{AK_p k_s}{\tau}}, \qquad \zeta = \frac{1}{2\sqrt{\tau A K_p k_s}}, \qquad \sigma = \zeta\omega_n = \frac{1}{2\tau}. \tag{4.3}$$

Therefore $K_p < 1/(4\tau A k_s \zeta_{\min}^2) = 0.0181544$ for nominal overshoot compliance. The added reporting targets are 60 s nominal and 75 s under drag uncertainty, using a 2% reference band. They supplement the four official requirements. Bias and signal scaling must be calibrated for ideal tracking.

4.1.3 Gain Selection (Root Locus)

The open-loop poles are 0 and $-1/\tau = -0.1234568$; there are no finite zeros. The real-axis locus lies between the poles, and its two asymptotes have angles $\pm 90°$ about centroid $-1/(2\tau)$. Differentiating $K_p(s) = -s(\tau s + 1)/(Ak_s)$ gives

$$\frac{dK_p}{ds} = 0 \Rightarrow s_b = -\frac{1}{2\tau} = -0.0617284, \quad K_{p,b} = \frac{1}{4\tau A k_s} = 0.00232489. \tag{4.4}$$

The supplied gain $K_p = 0.01432$ is above breakaway and below the nominal overshoot limit. Since the complex branches have fixed real part -0.0617284, a 30 s decay-envelope target ($\sigma \approx 4/30$) cannot be imposed by proportional gain alone. Detailed angle and magnitude checks are in Appendix E; the root-locus code folder reproduces the method.

4.1.4 Design Summary

Substituting $K_p = 0.01432$ gives $q = 14.2748(0.01432)(0.93) = 0.1901061$ and

$$G_{cl}(s) = \frac{0.1901061}{8.1s^2 + s + 0.1901061} = \frac{0.02346989}{s^2 + 0.1234568s + 0.02346989}. \tag{4.5}$$

The roots and response quantities in Table 4.1 are calculated from Equation (4.3) and checked against Figure 4.1.

Table 4.1: Manual nominal-controller calculations.

Quantity	Calculation	Result
Natural frequency	$\sqrt{0.1901061/8.1}$	0.153199 rad/s
Damping ratio	$1/[2\sqrt{8.1(0.1901061)}]$	0.402930
Damped frequency	$\omega_n\sqrt{1-\zeta^2}$	0.140212 rad/s
Closed-loop poles	$-\sigma \pm j\omega_d$	$-0.0617284 \pm j0.140212$
Overshoot	$100e^{-\pi(0.402930)/\sqrt{1-0.402930^2}}$	25.0804%
Peak time	$\pi/0.140212$	22.4060 s
Peak altitude	$5(1 + 0.250804)$	6.25402 m

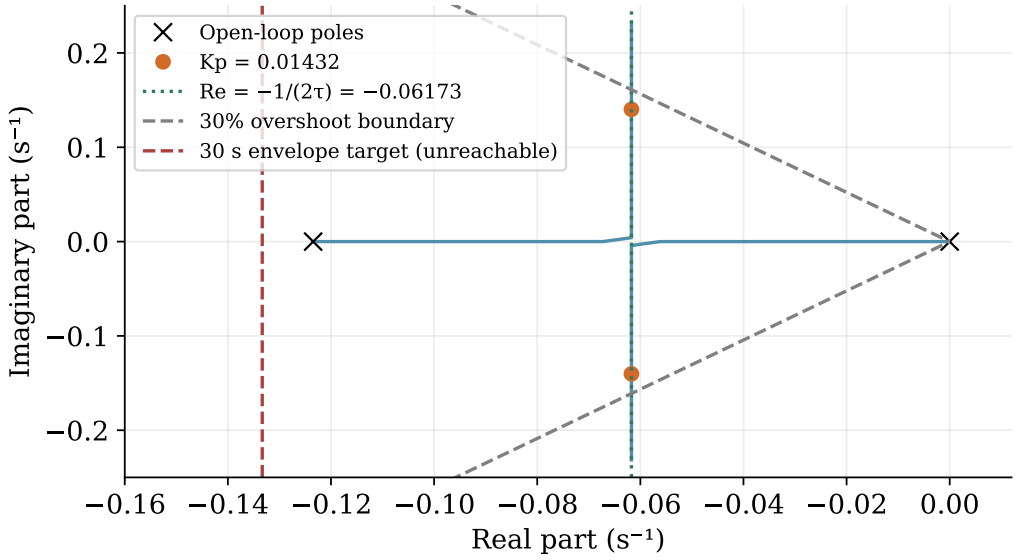


Figure 4.1: Proportional root locus. The selected poles satisfy the 30% overshoot boundary; the 30 s envelope target is beyond the attainable complex branches. Code: Python; MATLAB. Data: results.

The exact step response is

$$y(t) = 5\left[1 - e^{-\sigma t}\left(\cos\omega_d t + \frac{\sigma}{\omega_d}\sin\omega_d t\right)\right]. \tag{4.6}$$

The usual $4/\sigma = 64.8$ s settling approximation is conservative here. The envelope bound $-\ln[0.02\sqrt{1-\zeta^2}]/\sigma = 64.81$ s and the actual final 2% crossing, 54.89 s, are different quantities.

4.2 Simulink Model and Results

4.2.1 Block Diagram Implementation

Figure 4.2 is the supplied Simulink diagram. It uses 0.93 V/m reference/sensor gains, $K_p = 0.01432$, hover addition/subtraction, a velocity lag and position integrator. The reconstruction sets command limits to 0–5 V and sensor limits to 0–10 V.

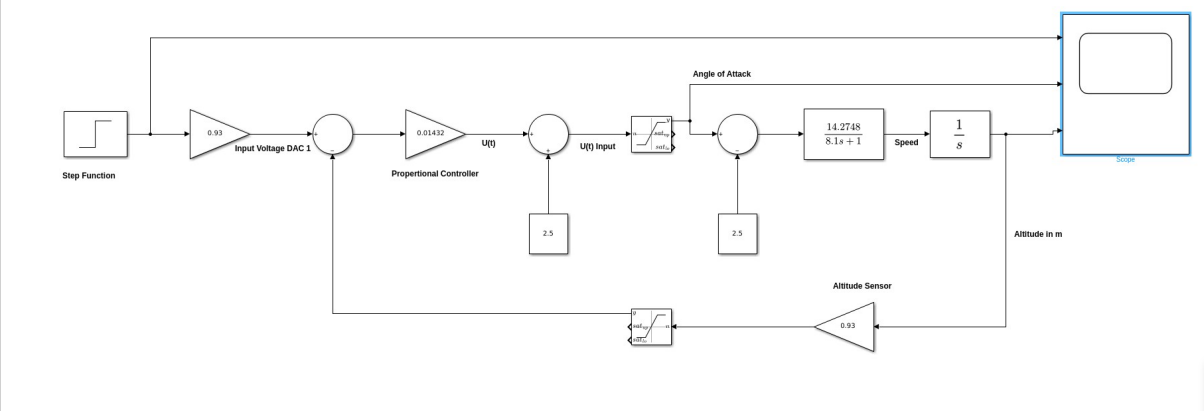


Figure 4.2: Supplied Simulink architecture. Native model-building and execution scripts: Simulink folder.

4.2.2 Comparison with the Supplied Simulation

The supplied scope starts at zero altitude with a 2.5 m reference, changing to 5 m at about 2.5 s. Python reproduces that same reference sequence in Figure 4.3; it is not the isolated 0-to-5 m design step. The altitude RMSE against that prediction is approximately 0.0115 m.

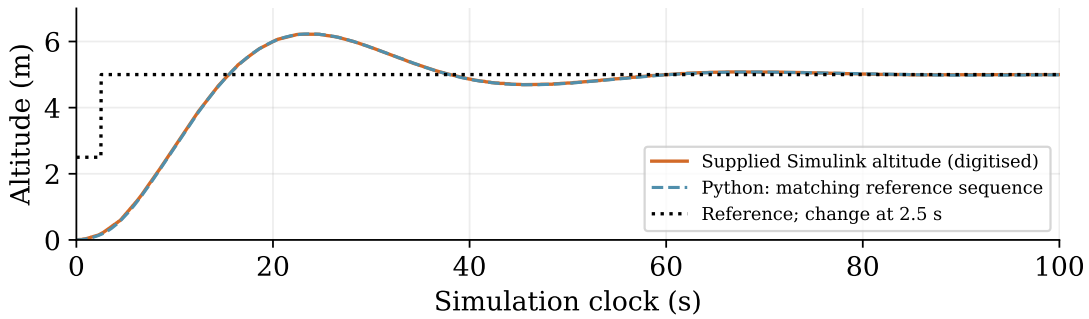


Figure 4.3: Supplied Simulink altitude, replotted and compared with Python for the same reference sequence. Code: Python; MATLAB. Data: results.

The supplied curve peaks at ≈ 6.226 m (24.51% above 5 m); the matched model predicts 6.231 m and maximum command 2.56441 V. For the isolated 5 m step, $u(0^+) = 2.5 + 0.01432(0.93)(5) = 2.56659$ V. Full settings and nominal results are in Appendix C.

4.2.3 Controller Hardware Implementation

Figure 4.4 shows the revised KiCad controller with its bias and DAQ connections. The physical three-stage circuit was connected in this configuration during the controller test. Capacitors and unused offset-pin graphics are omitted from the drawing. The board photograph remains in Appendix A.

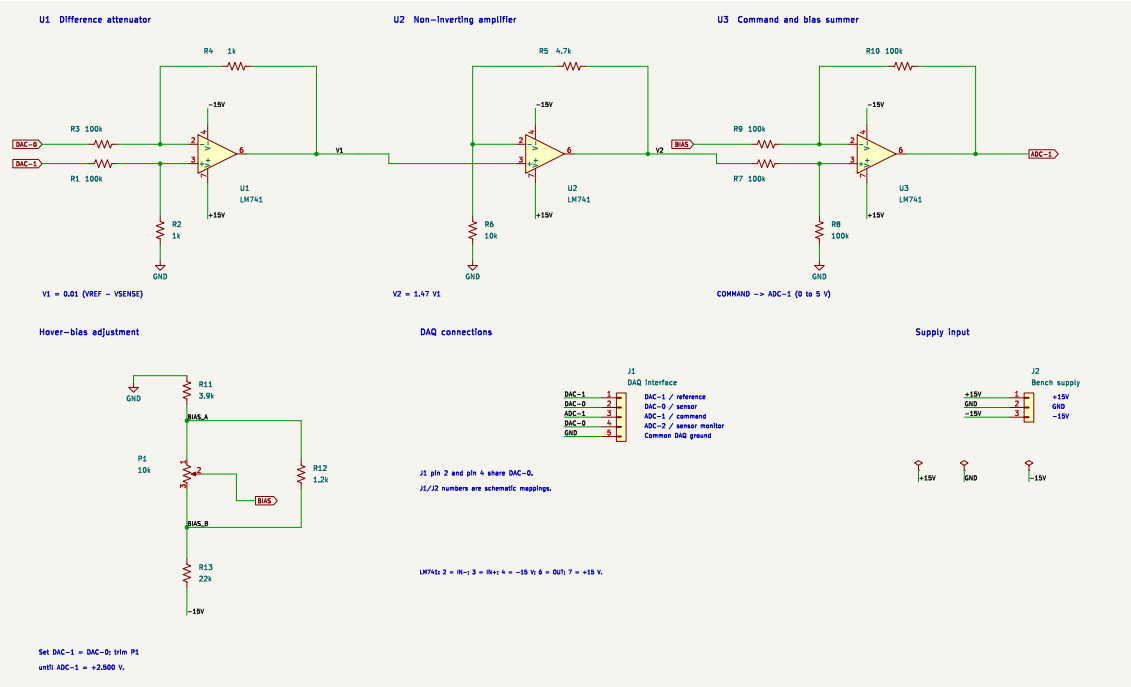


Figure 4.4: Three-stage controller with bias and DAQ connections. Editable source: KiCad folder.

Table 4.2: Controller component values and stage calculations.

Stage	Values	Function
U1	$R_1 = R_3 = 100$ kΩ; $R_2 = R_4 = 1$ kΩ	$V_1 = 0.01(r_v - y_v)$.
U2	$R_5 = 4.7$ kΩ; $R_6 = 10$ kΩ	$V_2 = 1.47V_1$.
U3	R_7–$R_{10} = 100$ kΩ	Loaded gain $g_3 = 0.974280$.
Bias	$R_{11} = 3.9$ kΩ, $R_{12} = 1.2$ kΩ, $R_{13} = 22$ kΩ, P1 = 10 kΩ	R_{11} to ground; R_{13} to −15 V.

All three LM741 stages use pin 7 for +15 V and pin 4 for −15 V, with common DAQ ground. Loading of the unbuffered bias network must be included:

$$u_0 = -\frac{100}{100 + 5.423001}(-2.635575) = 2.500000\,\mathrm{V}, \tag{4.7}$$

$$K_{p,\mathrm{circuit}} = 0.01(1.47)\frac{1 + 100/(100 + 5.423001)}{2} = 0.0143219. \tag{4.8}$$

Full loading and interface calculations are in Appendix D; test adjustment is in Section 5.

Chapter 5

Controller Tests

5.1 Connected Test Setup and Adjustment

The complete physical circuit was connected as in Figure 4.4: DAC-1 for reference, DAC-0 for sensor feedback, ADC-1 for command and ADC-2 for sensor monitoring. The common-ground ±15 V supply powered all three stages. The physical controller operated in closed loop with the simulated plant.

The physical potentiometer was adjusted until the input was approximately 2.565 V to improve setpoint tracking and minimise overshoot. Its exact measurement instant was not logged. The value is close to the model's initial 5 m step command, 2.56659 V, and the reconstructed Simulink maximum, 2.56441 V. It does not replace the nominal 2.500 V zero-error hover term; Appendix E.9 gives the full calculation.

5.2 Experimental Results and Comparison

At 86.10 s the display gives reference 5.033 m, height 4.897 m and command 2.498 V. This terminal command and the reported potentiometer adjustment are separate observations (Appendix B). Figure 5.1 replots the recorded controller response in Python.

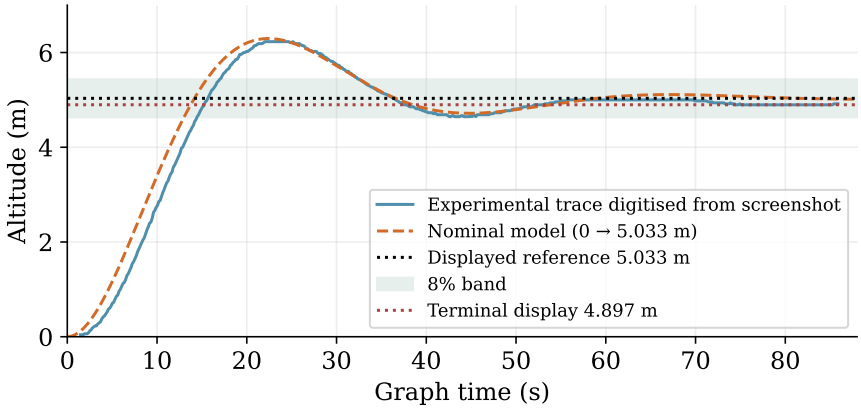


Figure 5.1: Connected controller response and nominal prediction for the same 5.033 m reference. Code: Python; MATLAB. Data: results.

The approximate peak is 6.229 m at 22.02 s. From the displayed reference and final height,

$$\eta_{\text{track}} = 100\left(1 - \frac{|5.033 - 4.897|}{5.033}\right) = 97.30\%, \tag{5.1}$$

$$M_{p,\text{ref}} \approx 100\frac{6.22917 - 5.033}{5.033} = 23.77\%. \tag{5.2}$$

5.3 Performance Analysis

Table 5.1 compares model, supplied Simulink and connected-controller evidence. The Simulink startup sequence is matched separately in Figure 4.3; image-derived values are approximate.

Table 5.1: Model, supplied Simulink and connected-controller comparison.

Metric	Nominal calculation / Python	Supplied Simulink	Connected circuit
Reference	0 to 5 m	2.5 to 5 m	Near 0 to 5.033 m
Peak altitude	6.25402 m	≈ 6.226 m	≈ 6.229 m
Peak clock time	22.405 s	≈ 24.0 s	≈ 22.02 s
Peak above final reference	25.0804%	≈ 24.51%	≈ 23.77%
Terminal tracking	100% ideal	≈ 99.98%	97.30%
2% band entry	54.89 s	≈ 56.05 s on source clock	Not established

The physical terminal height lies in the 8% reference band (4.63036–5.43564 m), outside the 2% band (4.93234–5.13366 m). Overshoot normalised to terminal height is 27.20%; exact physical settling remains unconfirmed. The Simulink clock includes a reference change at 2.5 s and initial motion from zero.

Table 5.2: Evaluation against the laboratory requirements.

Criterion	Evidence	Evaluation
Tracking $> 92\%$	97.30%; error 0.136 m.	Pass: retained nominal test.
Overshoot $< 30\%$	Python 25.08%; experiment 23.77%.	Pass: nominal responses.
Aerodynamics ±10%	Overshoot 21.15–29.45%.	Pass: drag-only predictions.
Components ±10%	Corner error 48.79%; overshoot 38.95%.	Fail: independent tolerances.

5.4 Discussion

Nominal peaks agree closely, but bias and unequal reference/feedback scaling can change tracking. With recalibrated hover and stage coefficients from Appendix D,

$$K_f = a_f g_2 g_3, \qquad y_\infty / x_\infty = a_r / a_f. \tag{5.3}$$

The 2,048 loop-resistor/drag corners fail robustness; joint scalar drag/gain variation also gives 31.42% overshoot. A +2 mV input bias predicts +0.150 m offset. Detailed uncertainty and correction calculations are in Appendix E; these are predictions, not additional physical trials.

Chapter 6

Conclusion and Recommendations

6.1 Conclusion

The laboratory workflow established a physical model, identified its parameters, evaluated a proportional controller and compared its predicted response with retained hardware evidence. Differentiating altitude gave the velocity model $G_v(s) = 14.2748/(8.1s + 1)$; integration gives the position plant $G_y(s) = 14.2748/[s(8.1s + 1)]$. Sensor calibration gave $k_s = 0.93$ V/m.

The three-stage analogue controller includes the 2.5 V hover bias. Accounting for loading in the bias network gives $K_p = 0.0143219$, consistent with the adopted 0.01432 gain. Root-locus and time-response calculations predict 25.08% overshoot and 54.89 s settling in the 2% reference band. The retained experiment gives 97.30% tracking accuracy and approximately 23.77% overshoot about the reference, satisfying the nominal tracking and overshoot criteria.

The robustness evaluation identifies a limitation: drag uncertainty alone passes, but joint drag/gain variation and independent resistor tolerances can exceed the requirements. One retained trial also cannot establish experimental repeatability. These findings support the nominal design and specify the changes needed for a stronger tolerance claim.

6.2 Recommendations

1. **Match the reference and feedback ratios.** Use matched resistors or trim U1 to below 1% ratio mismatch; measure both signal slopes.
2. **Add overshoot margin.** Replace both 1 kΩ U1 resistors with matched 680 Ω values. The proposed loaded gain is 0.00974 and worst scalar-case overshoot is 23.44%.
3. **Recalibrate the loaded hover command.** Trim to 2.500 V after component changes. At the revised gain, the 5 m bias allowance is 3.623 mV.
4. **Retain repeat tests and raw logs.** Record independent identification and controller trials, command headroom and defined disturbance tests to assess repeatability and robustness.

6.3 Final Remarks

The report documents the complete design process and the limits of the supplied evidence. Proposed circuit changes require physical retesting. Worked derivations are in Appendix E; Appendix H links each supporting folder and the complete GitHub repository.

References

[1] University of Cape Town, Department of Electrical Engineering, *EEE3094S Lab 2: Proportional Control*, 2026. Supplied laboratory brief and rubrics.

[2] UCT, *EEE3094S Lab 2/3 LaTeX Example*, supplied template. N. Mathebula, *Lab 2 Report*, 2025, supplied structure example.

[3] MathWorks, *stepinfo*. Online documentation.

[4] MathWorks, *rlocus*. Online documentation.

[5] MathWorks, *exportgraphics*. Vector PDF and image export documentation.

[6] MathWorks, *sim* and *To Workspace*. Simulation execution; logged output documentation.

Appendix A: Photographs of the Final Controller

Figure A.1 documents the soldered Veroboard implementation described in Section 4.2.3. The board has three eight-pin op-amp packages and a blue bias trimmer. The photograph records the final physical assembly; it does not establish every underside connection or measured resistance.

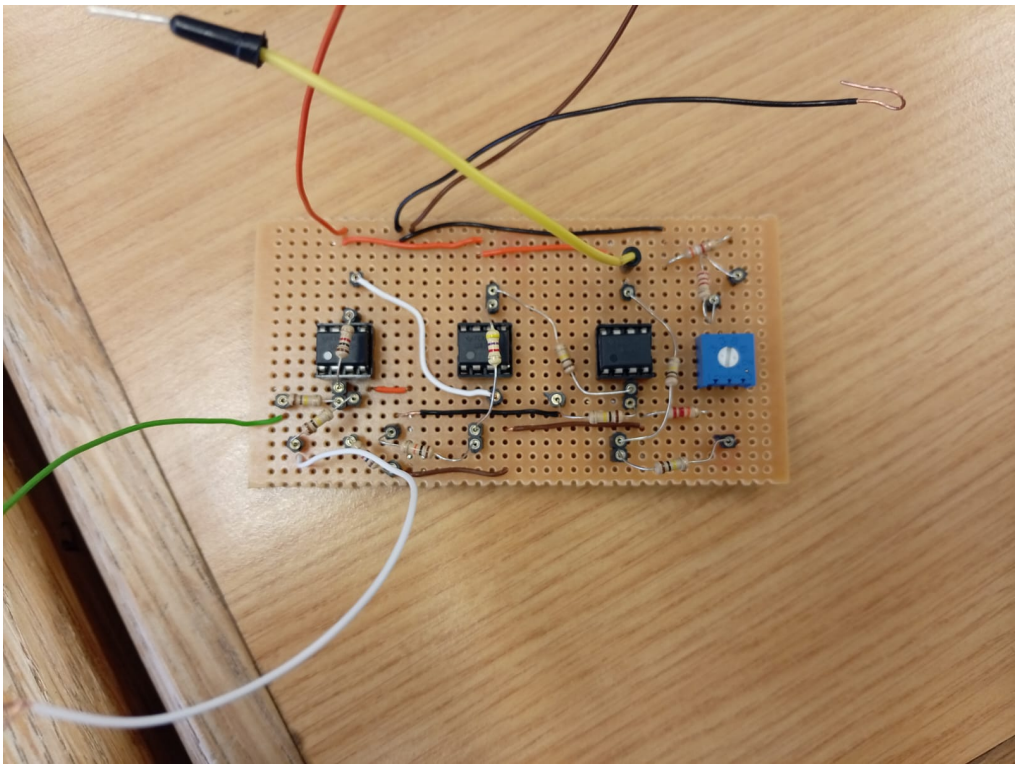


Figure A.1: Supplied final Veroboard photograph. The three op-amp stages and adjustable bias trimmer are visible.

The circuit design specifies 741-type devices and bipolar supply rails. The current KiCad drawing contains no capacitors and hides unused offset-pin graphics. Device markings and underside wiring are not independently established by this top-side photograph. No photographs of handwritten calculations are used in this report.

Appendix B: Experimental Demonstration Record

The original controller-test display is retained in the Session 2 evidence folder. The altitude trace is replotted with Python in Figure 5.1. The original application screenshot below is supporting demonstration evidence; the main numerical comparison is the regenerated Python plot.

The full controller was connected during this test, as confirmed in Section 5. The student also reported adjusting the physical potentiometer until the input was approximately 2.565 V. This recollection is separate from the later terminal readouts below.

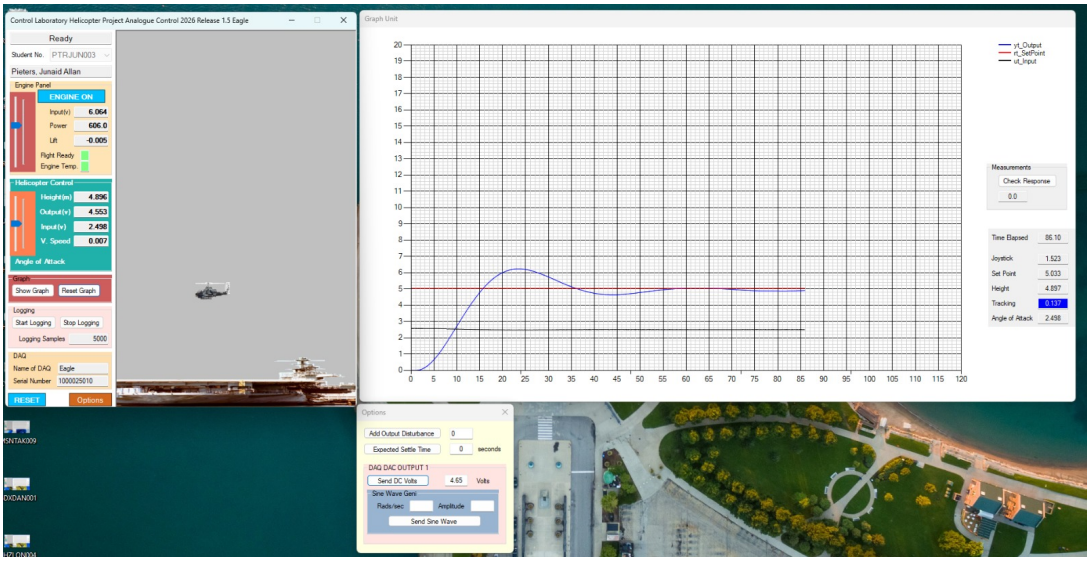


Figure B.1: Original controller-demonstration screenshot from the laboratory application, retained as required evidence. The numerical response is replotted in Figure 5.1.

The numerical values transcribed in Table B.1 support the tracking and sensor checks.

Table B.1: Numerical readouts transcribed from the retained test display.

Field	Displayed value
Student number	PTRJUN003
Set point	5.033 m
Height	4.897 m
Tracking error	0.137 m
Angle of attack / command	2.498 V
Height (left panel)	4.896 m
Sensor output (left panel)	4.553 V
Elapsed time	86.10 s

The 0.001 m differences between the readouts are consistent with rounding or asynchronous display updates. The assessor's marking sheet and signature were not supplied, so an awarded demonstration mark is not claimed.

Appendix C: Simulink Configuration and Reproduction

The supplied model diagram appears in Figure 4.2; its results are replotted and compared in Figure 4.3. The unaltered scope image is available through the original evidence folder. The digitised samples are stored separately in results; they are not raw solver logs.

Table C.1: Simulink parameters and test conditions.

Block / condition	Setting
Reference and sensor gain	0.93 V/m each
Proportional controller	0.01432 V/V
Hover add/subtract	2.500 V in the nominal model
Velocity transfer function	$14.2748/(8.1s + 1)$
Position integrator	Zero initial altitude
Command / sensor saturation	Explicit reconstruction limits: 0–5 V / 0–10 V
Supplied scope sequence	Reference 2.5 m initially, then 5 m at about 2.5 s; zero initial altitude
Isolated design sequence	0 to 5 m at $t = 0$

The screenshot does not display the saturation block parameter dialogs; the limits in the table are explicit settings in the reconstruction. The original native `.slx` was not supplied. The MATLAB/Simulink scripts build a native model, execute both sequences and export logged responses when run in MATLAB with Simulink.

```
addpath(genpath('matlab'));
P = lab_parameters();
run_simulink_tests(P);
plot_report_figures(P);
```

The current report figures were rendered with Python. MATLAB reproduction code is provided; it has not been executed in this environment. The supplied Simulink scope is retained evidence of the student's simulation. Export and logged-output APIs follow the official documentation [5, 6].

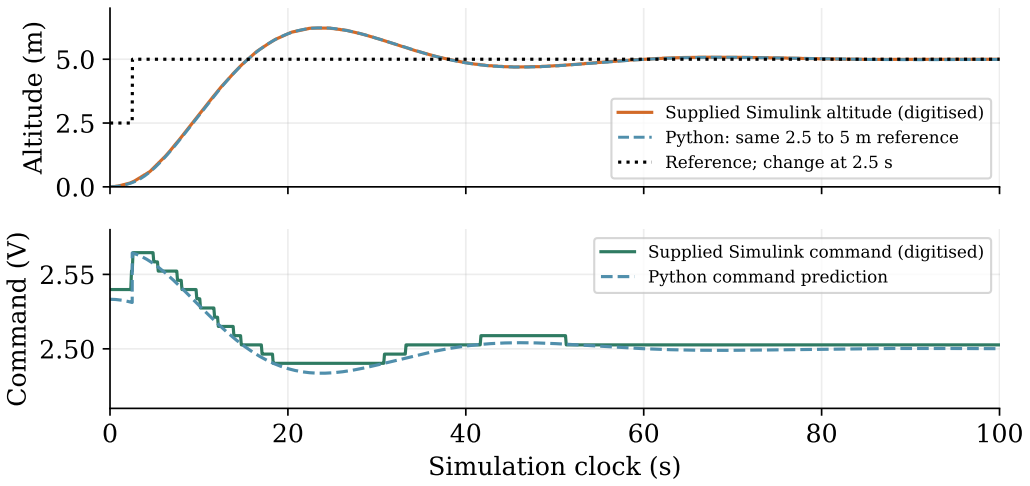


Figure C.1: Full altitude and command comparison for the supplied Simulink sequence. Code: Python; MATLAB. Data: results.

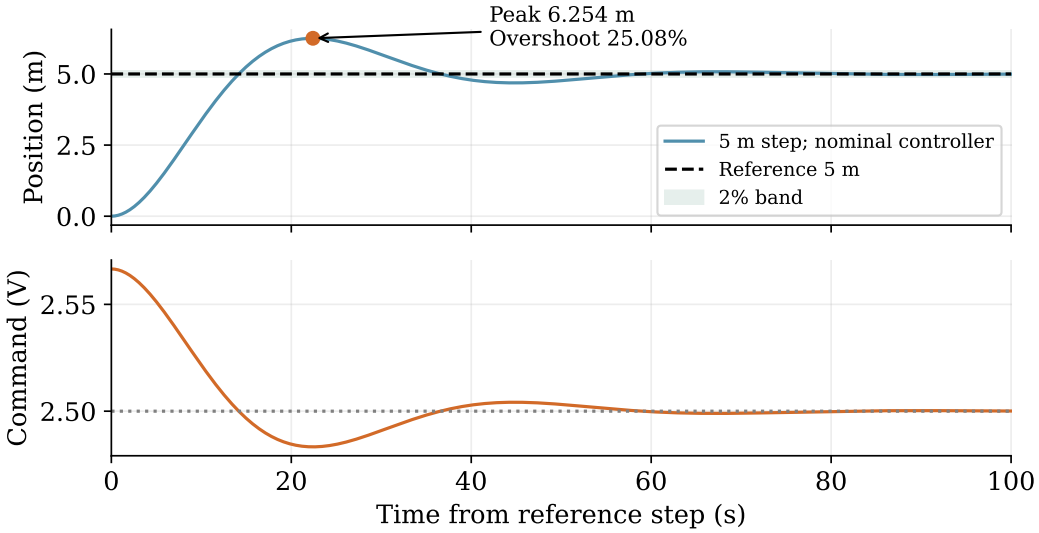


Figure C.2: Isolated nominal 0-to-5 m design test, showing altitude and command. This is a distinct reference sequence from the supplied scope. Code: Python; MATLAB. Data: results.

For this test the peak is 6.25402 m, overshoot 25.0804%, peak time 22.405 s and final 2% reference-band entry 54.89 s. The maximum command is 2.56659 V and the minimum 2.48330 V; neither reaches the command limits.

Appendix D: Complete Circuit and Interface

D.1 Signal Stages

The complete revised KiCad circuit is in Figure 4.4, with an editable project and BOM in the KiCad folder. With ideal op-amp inputs, the U1 positive divider is $V_+ = r_v R_2/(R_1 + R_2)$. Applying KCL at the inverting input gives

$$V_1 = \left(1 + \frac{R_4}{R_3}\right) \frac{R_2}{R_1 + R_2} r_v - \frac{R_4}{R_3} y_v, \tag{D.1}$$

$$= (1 + 1/100)(1/101) r_v - (1/100) y_v = 0.01(r_v - y_v), \tag{D.2}$$

$$V_2 = \left(1 + \frac{R_5}{R_6}\right) V_1 = (1 + 4.7/10) V_1 = 1.47 V_1. \tag{D.3}$$

U3's positive divider gives $V_+ = V_2 R_8/(R_7 + R_8) = V_2/2$. Its bias loading is derived in Section D.3.

All three op-amps use pin 7 for +15 V, pin 4 for −15 V, pin 2 for the inverting input, pin 3 for the non-inverting input and pin 6 for output. Pins 1 and 5 (offset null) and pin 8 (NC) are unused. The hidden offset pins remain in the native symbol with their physical assignments; only their visible graphics and cross marks are omitted.

D.2 Bias Network and DAQ Interface

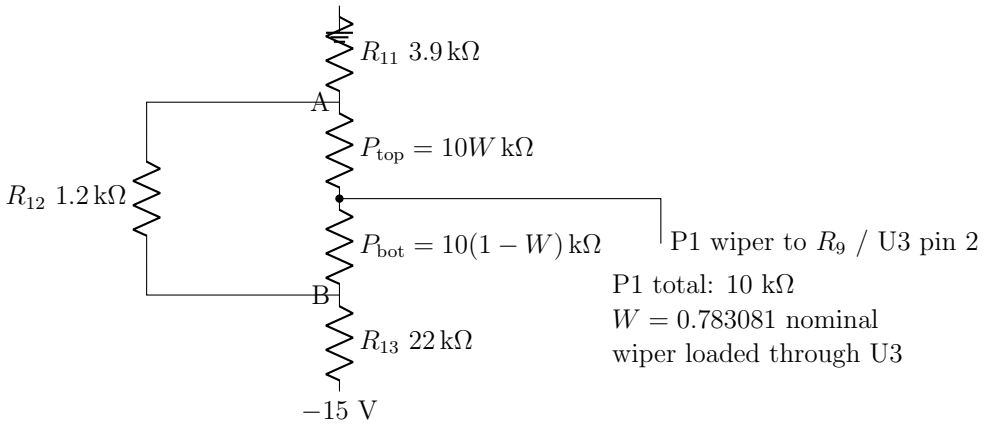


Figure D.1: Bias potentiometer represented by its two segments, shunt resistor and end resistors. No capacitors are included in the revised KiCad drawing.

Table D.1: DAQ and supply connections.

Connection	Controller node
DAC-1	r_v, U1 reference branch through R_1.
DAC-0	y_v, U1 sensor branch through R_3.
ADC-2	Sensor monitor at y_v.
ADC-1	Command u, U3 pin 6; permitted plant range 0–5 V.
Ground	Shared by DAQ, bias network and bench supply.
±15 V	Op-amp rails; these do not define the permitted plant command.

The LTspice source and supplied circuit image place R_{11} at ground; the current native KiCad schematic agrees. The theoretical hover calibration sets $r_v = y_v$ and trims the loaded command to 2.500 V. This theoretical condition is distinct from the reported approximately 2.565 V input adjustment during the connected controller test. J1/J2 pin numbering is a logical channel/supply map rather than a verified physical connector terminal order.

D.3 Bias Loading and Stage Gain Calculations

With the wiper unloaded, $R_P = P1 \parallel R_{12} = 10 \parallel 1.2 = 1.0714286$ kΩ. The end-to-end current and pot-end voltages are

$$I = \frac{-15}{3.9 + 1.0714286 + 22} = -0.556143\,\text{mA}, \tag{D.4}$$

$$V_A = I(3.9) = -2.168962\,\text{V}, \qquad V_B = I(3.9 + 1.0714286) = -2.764831\,\text{V}, \tag{D.5}$$

$$V_{\text{th}} = V_A + W(V_B - V_A). \tag{D.6}$$

Set the supply sources to zero and inject a 1 A test current at the wiper. The node-conductance equations for A, B and the wiper are

$$\begin{bmatrix} G_{11} + G_{12} + G_t & -G_{12} & -G_t \\ -G_{12} & G_{13} + G_{12} + G_b & -G_b \\ -G_t & -G_b & G_t + G_b \end{bmatrix} \begin{bmatrix} V'_A \\ V'_B \\ V'_w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \tag{D.7}$$

where $G_i = 1/R_i$, $G_t = 1/(WP1)$ and $G_b = 1/[(1-W)P1]$. The test voltage V'_w gives R_{th}. Solving the hover equation

$$-\frac{R_{10}}{R_9 + R_{\text{th}}(W)} V_{\text{th}}(W) = 2.5 \tag{D.8}$$

gives $W = 0.783080594$, $R_{\text{th}} = 5423.0014\ \Omega$ and $V_{\text{th}} = -2.6355750$ V. The loaded zero-error wiper is -2.500 V.

For U3, define $r = R_{10}/(R_9 + R_{\text{th}}) = 0.94855960$. Since the non-inverting divider gives $V_+ = V_2/2$, virtual short and KCL give

$$u = (1 + r)V_+ - rV_{\text{th}} = \frac{1 + r}{2} V_2 - rV_{\text{th}}, \tag{D.9}$$

$$g_3 = \frac{1 + 0.94855960}{2} = 0.97427980, \tag{D.10}$$

$$K_p = \left(\frac{1}{100}\right)\left(1 + \frac{4.7}{10}\right)(0.97427980) = 0.014321913. \tag{D.11}$$

The ideal buffered result 0.0147 differs by 2.57%, while the selected model value 0.01432 differs from the loaded calculation by only 0.0134%. These are circuit-model calculations; a measured input/output slope of the final board was not supplied.

Appendix E: Detailed Calculations

E.1 Model Derivation and Signal Definitions

Table E.1: Model notation and units.

Symbol	Description	Unit
$x, y, v = \dot{y}$	Reference altitude, altitude, vertical velocity	m, m, m/s
f_t, g, d_a	Mass-normalised thrust, gravity, disturbance	$\mathrm{m/s^2}$
b, k_T	Drag coefficient; local voltage-to-thrust slope	$\mathrm{s^{-1}}$; $\mathrm{m/(s^2\ V)}$
$u, u_h, \delta u$	Command, hover command, $u - u_h$	V
A, τ	Velocity gain; velocity time constant	m/(s V); s
k_s	Position-to-voltage coefficient	V/m
r_v, y_v, e_v	Reference, sensor, error voltage	V
K_p	Proportional voltage-error gain	V/V

Start from $\ddot{y} = f_t - b\dot{y} - g + d_a$, with $f_t = g + k_T(u - u_h)$. For $d_a = 0$, zero initial conditions and $v = \dot{y}$,

$$\dot{v} + bv = k_T \delta u, \tag{E.1}$$

$$(s + b)V(s) = k_T \Delta U(s), \tag{E.2}$$

$$G_v(s) = \frac{k_T}{s + b} = \frac{k_T/b}{s/b + 1} = \frac{A}{\tau s + 1}, \tag{E.3}$$

$$G_y(s) = \frac{G_v(s)}{s} = \frac{A}{s(\tau s + 1)}. \tag{E.4}$$

The identified parameters give $b = 1/8.1 = 0.123456790\ \mathrm{s^{-1}}$ and $k_T = 14.2748/8.1 = 1.762320988\ \mathrm{m/(s^2\ V)}$. The hover voltage does not imply $k_T = g/u_h$.

For negative feedback, $E_y = X - Y$ and $L = K_p k_s G_y$:

$$\frac{E_y(s)}{X(s)} = \frac{1}{1 + L(s)} = \frac{\tau s^2 + s}{\tau s^2 + s + AK_p k_s}. \tag{E.5}$$

For a step $X = x_0/s$, the final-value theorem gives $e_\infty = \lim_{s \to 0} sE_y(s) = 0$. For a ramp $X = a_r/s^2$,

$$e_\infty = \frac{a_r}{AK_p k_s}, \qquad K_v = AK_p k_s = 0.190106076. \tag{E.6}$$

A parabolic reference has unbounded steady error. A constant acceleration disturbance d_0 instead gives $y_\infty - x_\infty = d_0/(k_T K_p k_s)$. For an input bias, $d_0 = k_T \delta u_b$, so $\delta y = \delta u_b/(K_p k_s)$.

E.2 Identification and Timing

Table E.2 gives selected original samples and derived velocities. The finite manual step is evident in the input column. Endpoint times are listed for consistency with the CSV; each derivative physically belongs to its interval midpoint.

Table E.2: Selected identification samples and backward-difference results.

t [s]	u [V]	y_v [V]	y [m]	v_i [m/s]
15.7	2.495	0.000	0.000	0.00
15.8	2.908	0.000	0.000	0.00
15.9	4.607	0.000	0.000	0.00
16.0	5.000	0.034	0.037	0.37
16.9	5.000	2.098	2.256	4.08
17.0	5.000	2.513	2.703	4.47
23.9	5.000	10.000	104.823	22.36
24.0	5.000	10.000	107.076	22.53
24.1	5.000	10.000	109.345	22.69

$$v(24.0) = (107.076 - 104.823)/0.1 = 22.53\,\mathrm{m/s}, \tag{E.7}$$

$$v(24.1) = (109.345 - 107.076)/0.1 = 22.69\,\mathrm{m/s}, \tag{E.8}$$

$$\bar{v}_\infty = \frac{1}{404} \sum_{t_i \geq 120} \frac{y_i - y_{i-1}}{0.1} = 35.68747525\,\mathrm{m/s}, \tag{E.9}$$

$$A = (35.68747525 - 0)/2.5 = 14.27499010\,\mathrm{m/(s\,V)}, \tag{E.10}$$

$$v_{63} = 0.632(35.68747525) = 22.55448436\,\mathrm{m/s}, \tag{E.11}$$

$$t_{63} = 24.0 + 0.1\frac{22.55448436 - 22.53}{22.69 - 22.53} = 24.01530272\,\mathrm{s}, \tag{E.12}$$

$$\tau_{63} = 24.01530272 - 15.9 = 8.11530272\,\mathrm{s}. \tag{E.13}$$

The exponential fit minimises $\sum_{17 \leq t_i \leq 60} [v_i - V_\infty(1 - e^{-(t_i - t_0)/\tau})]^2$. Its outputs are $V_\infty = 35.68747059$ m/s, $\tau = 8.10393362$ s and $t_0 = 15.91583783$ s. These two methods corroborate rounding to $\tau = 8.1$ s. They do not create two experimental trials.

The sensor least-squares fit in Equation (3.5) excludes zero and clipped pairs. The check $k_s(2.703) = 0.93(2.703) = 2.51379$ V agrees with the recorded 2.513 V within rounding.

E.3 Root-Locus Geometry and Gain Bound

Step 1 – Form the loop without the variable gain. Include the sensor coefficient explicitly:

$$G_0(s) = k_s G_y(s) = \frac{14.2748(0.93)}{s(8.1s+1)} = \frac{1.638958519}{s(s+0.123456790)}. \tag{E.14}$$

Step 2 – Locate poles and asymptotes. The two poles are $p_1 = 0$ and $p_2 = -0.123456790$; there are no finite zeros. The real-axis locus is between the poles. With $n - m = 2$,

$$\sigma_a = \frac{p_1 + p_2}{2} = -0.061728395, \qquad \theta_\ell = \frac{(2\ell+1)180^\circ}{2} = 90^\circ, 270^\circ. \tag{E.15}$$

Step 3 – Calculate breakaway. From $1 + K_p G_0(s) = 0$,

$$K_p(s) = -\frac{s(8.1s+1)}{14.2748(0.93)}, \tag{E.16}$$

$$\frac{dK_p}{ds} = -\frac{16.2s+1}{13.275564} = 0 \quad \Rightarrow \quad s_b = -\frac{1}{16.2} = -0.061728395, \tag{E.17}$$

$$K_{p,b} = \frac{1}{4(8.1)(14.2748)(0.93)} = 0.002324888. \tag{E.18}$$

Step 4 – Apply the overshoot bound. Solving $0.30 = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$ gives

$$\zeta_{\min} = \frac{-\ln 0.30}{\sqrt{\pi^2 + (\ln 0.30)^2}} = 0.357857131. \tag{E.19}$$

The characteristic polynomial is $s^2 + s/8.1 + (14.2748)(0.93)K_p/8.1$. Matching coefficients gives $\omega_n^2 = AK_p k_s/\tau$ and $2\zeta\omega_n = 1/\tau$, hence

$$K_p < \frac{1}{4\tau A k_s \zeta_{\min}^2} = \frac{1}{4(8.1)(14.2748)(0.93)(0.357857131)^2} = 0.018154432. \tag{E.20}$$

The adopted $K_p = 0.01432$ lies between breakaway and the nominal overshoot limit. The complex branches have fixed decay rate $\sigma = 1/(2\tau) = 0.061728395\ \mathrm{s}^{-1}$; they cannot reach $\sigma = 4/30 = 0.133333\ \mathrm{s}^{-1}$ by changing K_p alone.

E.4 Selected Gain and Pole Calculations

Step 5 – Check the adopted gain. This value is retained from the supplied controller design; the calculations test its performance. Substitution gives

$$q = AK_p k_s = (14.2748)(0.01432)(0.93) = 0.190106076, \tag{E.21}$$

$$G_{cl}(s) = \frac{0.190106076}{8.1s^2 + s + 0.190106076}, \tag{E.22}$$

$$\omega_n = \sqrt{q/\tau} = \sqrt{0.190106076/8.1} = 0.153198845\,\text{rad/s}, \tag{E.23}$$

$$\zeta = \frac{1}{2\sqrt{\tau q}} = \frac{1}{2\sqrt{8.1(0.190106076)}} = 0.402929900, \tag{E.24}$$

$$\omega_d = \sqrt{\omega_n^2 - \sigma^2} = 0.140212308\,\text{rad/s}, \tag{E.25}$$

$$s_{1,2} = \frac{-1 \pm \sqrt{1 - 4(8.1)(0.190106076)}}{2(8.1)} = -0.061728395 \pm j0.140212308. \tag{E.26}$$

The damping predicts $M_p = 100e^{-\pi(0.402929900)/\sqrt{1-0.402929900^2}} = 25.080375\%$. The next angle and magnitude checks verify that these poles lie on the proportional root locus.

E.5 Root-Locus Angle and Magnitude Checks

Step 6 – Verify the pole position. The locus for variable K_p uses $G_0(s) = k_s G_y(s) = (Ak_s/\tau)/[s(s + 1/\tau)]$, where $Ak_s/\tau = 1.638959\ \text{s}^{-2}$. At the selected upper pole $s_d = -0.0617284 + j0.1402123$,

$$\angle s_d = 113.76147^\circ, \qquad \angle(s_d + 1/\tau) = 66.23853^\circ, \tag{E.27}$$

$$\angle G_0(s_d) = -113.76147^\circ - 66.23853^\circ = -180^\circ, \tag{E.28}$$

$$K_p = \frac{1}{|G_0(s_d)|} = \frac{|s_d|\,|s_d + 1/\tau|}{Ak_s/\tau} = \frac{(0.153198845)^2}{1.63895852} = 0.01432. \tag{E.29}$$

Both the negative-feedback angle criterion and magnitude criterion are satisfied. The quadratic roots independently give the same pole pair.

E.6 Nominal Step and Settling Calculation

Step 7 – Evaluate the time response. Substituting into Equation (4.6),

$$y(t) = 5\left[1 - e^{-0.061728395t}\left(\cos(0.140212308t) + 0.44025\sin(0.140212308t)\right)\right]. \tag{E.30}$$

The first peak occurs at $t_p = \pi/\omega_d = 22.405969$ s. Since $\cos\pi = -1$ and $\sin\pi = 0$,

$$y(t_p) = 5[1 + e^{-0.061728395(22.405969)}] = 6.254019\,\text{m}, \tag{E.31}$$

so $M_p = (6.254019 - 5)/5 \times 100 = 25.080375\%$.

Step 8 – Calculate settling. The error envelope satisfies $|y-5|/5 \le e^{-\sigma t}/\sqrt{1-\zeta^2}$. Solving this bound for 2% gives

$$T_{\text{env}} = -\frac{\ln(0.02\sqrt{1-0.4029299^2})}{0.061728395} = 64.80975\,\text{s}. \tag{E.32}$$

Actual settling is found by evaluating the full response, locating the last sample for which $|y-5| > 0.1$ m, and taking the next sample. A 0.005 s grid gives 54.89 s. This definition agrees with reference-based settling calculations [3].

Step 9 – Check command headroom. Command headroom follows from $u = 2.5 + K_p k_s(5-y)$: the highest command is 2.566588 V at the step, and the lowest is $2.5 - 0.01432(0.93)(1.254019) = 2.483299$ V at the altitude peak. Neither reaches the 0 or 5 V limit.

E.7 Aerodynamic and Component Uncertainty

With fixed k_T, the uncertain characteristic polynomial is $s^2 + b's + k_T K_p k_s$. The worst scalar case has $b' = 0.9b_0$ and $K_p' = 1.1K_{p,0}$. Applying the required damping ratio gives

$$K_{p,0} < \frac{(0.9b_0)^2}{4k_T k_s \zeta_{\min}^2 (1.1)} = \frac{0.81}{1.1}(0.01815443) = 0.01336826. \tag{E.33}$$

At the supplied gain the worst-case damping is $(0.9/\sqrt{1.1})(0.4029299) = 0.34575$, and Equation (4.2) gives 31.42% overshoot. This explains why passing drag-only uncertainty is insufficient for the joint case.

For independent loop-resistor tolerances, use the U1 coefficients and U2/U3 gains defined in Section 4.2.3. Assign each of R_1–R_{10} either 0.9 or 1.1 times its nominal value, then use both drag extremes. There are $2^{10} \times 2 = 2048$ cases. The bias network is held at its nominal Thevenin resistance and zero-error output is recalibrated in each case. Table E.3 reports this optimistic subset.

Table E.3: Independent loop-resistor corner predictions.

Quantity	Range / worst value
Feedback gain K_f	0.009072–0.022784
Reference/feedback ratio a_r/a_f	0.672104–1.487864
Absolute DC tracking error	48.7864%
Overshoot about each case's own final altitude	38.9461%

For example, $R_1 = 0.9(100\,\text{k}\Omega)$, $R_2 = 1.1(1\,\text{k}\Omega)$, $R_3 = 1.1(100\,\text{k}\Omega)$ and $R_4 = 0.9(1\,\text{k}\Omega)$ give

$$a_f = 0.9/110 = 0.00818182, \quad \frac{a_r}{a_f} = \frac{(1 + 0.00818182)(1.1)/(90 + 1.1)}{0.00818182} = 1.487864. \tag{E.34}$$

Thus a 5 m reference tends to 7.43932 m even after hover recalibration. The type number cannot remove unequal reference and feedback scaling.

E.8 Practical Gain and Ratio Correction

The corner distributions in Figure E.1 separate dynamic overshoot from DC ratio error.

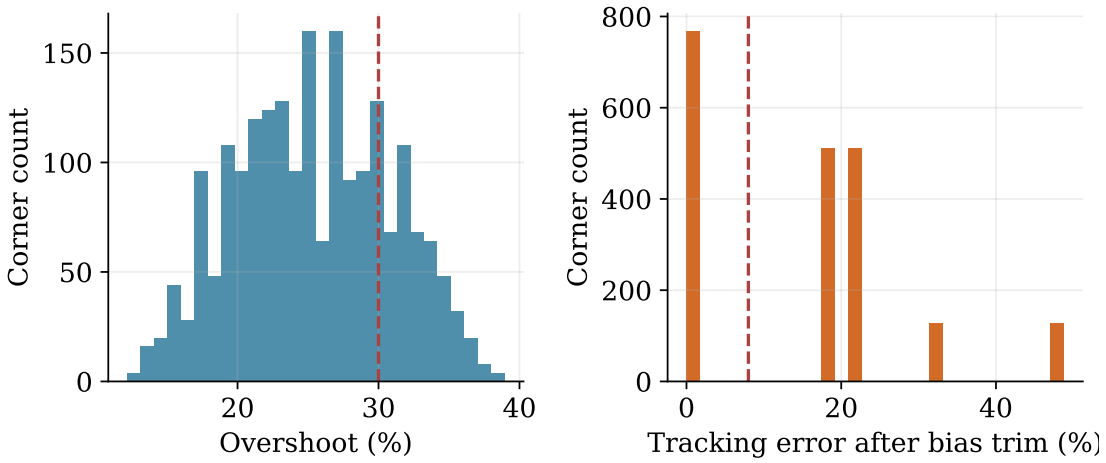


Figure E.1: Loop-resistor corners. Dashed limits are 30% overshoot and 8% tracking error. The histograms show corner counts, not probabilities. Code: Python; MATLAB. Data: results.

Failure in this subset rejects a guarantee for all components. Bias-network and op-amp variations can add error. After matching the U1 ratios, a practical gain correction is to replace both 1 kΩ resistors with matched 680 Ω values:

$$K_{p,\mathrm{new}} = \frac{680}{100000}(1.47)(0.97427980) = 0.009738901, \tag{E.35}$$

$$\zeta_{\mathrm{worst}} = \frac{0.9b_0}{2\sqrt{k_T(1.1K_{p,\mathrm{new}})k_s}} \approx 0.4193, \tag{E.36}$$

$$M_{p,\mathrm{worst}} \approx 23.44\%, \tag{E.37}$$

$$|\delta u_b|_{\max} = 0.08(5)(0.009738901)(0.93) = 3.623\,\mathrm{mV}. \tag{E.38}$$

This scalar check assumes matched reference and feedback ratios; it is not a guarantee for independent resistor corners. Lower gain increases damping but tightens the hover-bias allowance. The board must be recalibrated and retested after any change.

E.9 Reported Potentiometer Adjustment and Command Calculations

The complete physical three-op-amp controller was connected during the controller test. The student reported adjusting the physical potentiometer until the input was approximately 2.565 V, with the aim of tracking the setpoint with minimal overshoot. No time series or exact measurement instant accompanies that recollection. The retained terminal display independently records a 2.498 V command at 86.10 s.

For the isolated 0-to-5 m reference test, starting at zero altitude,

$$r_v(0^+) = k_s x = 0.93(5) = 4.650\,\mathrm{V}, \tag{E.39}$$

$$e_v(0^+) = r_v - y_v = 4.650 - 0 = 4.650\,\mathrm{V}, \tag{E.40}$$

$$\delta u(0^+) = K_p e_v = 0.01432(4.650) = 0.066588\,\mathrm{V}, \tag{E.41}$$

$$u(0^+) = u_h + \delta u = 2.500000 + 0.066588 = 2.566588\,\mathrm{V}, \tag{E.42}$$

$$u_{\mathrm{reported}} - u(0^+) = 2.565 - 2.566588 = -0.001588\,\mathrm{V}. \tag{E.43}$$

The difference in voltage magnitude is approximately 1.59 mV. This comparison does not establish that the reported observation was measured at the same instant or node as the model command.

For the supplied Simulink startup sequence, define the unit closed-loop response

$$h(t) = 1 - e^{-\sigma t}\left(\cos\omega_d t + \frac{\sigma}{\omega_d}\sin\omega_d t\right), \quad h(t) = 0 \text{ for } t < 0. \tag{E.44}$$

Superposition gives $y(t) = 2.5h(t) + 2.5h(t - 2.5)$ for a reference of 2.5 m initially and 5 m after 2.5 s. Consequently,

$$u(2.5^+) = 2.5 + 0.01432(0.93)\{5 - 2.5h(2.5)\} = 2.564405\,\mathrm{V}. \tag{E.45}$$

This maximum differs from the reported 2.565 V by approximately 0.595 mV. The original scope's voltage resolution is approximately 0.0124 V/pixel, so it cannot verify either millivolt difference.

The command during a reference error must be distinguished from the zero-error hover bias. If 2.565 V were instead held as a constant zero-error command, its 65 mV excess would predict

$$\delta y = \frac{2.565 - 2.500}{K_p k_s} = \frac{0.065}{0.01432(0.93)} = 4.88076\,\mathrm{m}. \tag{E.46}$$

That is not a negligible hover-bias change. Therefore the reported adjustment is documented as a practical test observation, rather than silently replacing the nominal 2.500 V hover term or the calculated controller gain. Changing the bias alone does not change the damping ratio or remove the proportional controller's settling limitation.

E.10 Experimental Metrics and Uncertainty

The digitised plot uses pixel bounds $x = 602 \ldots 1414$ for 0–120 s and $p_y = 60 \ldots 540$ for 20–0 m. Therefore

$$t = \frac{x - 602}{1414 - 602}(120), \qquad h = \frac{540 - p_y}{540 - 60}(20), \tag{E.47}$$

$$\Delta t = 0.147783\,\text{s/pixel}, \qquad \Delta h = 0.041667\,\text{m/pixel}. \tag{E.48}$$

The extracted peak is 6.229167 m at 22.019704 s. With the displayed reference, terminal height and zero starting altitude,

$$e_f = 5.033 - 4.897 = 0.136\,\text{m}, \tag{E.49}$$

$$\eta_{\text{track}} = 100(1 - 0.136/5.033) = 97.297834\%, \tag{E.50}$$

$$M_{p,\text{ref}} = 100(6.229167 - 5.033)/5.033 = 23.766475\%, \tag{E.51}$$

$$\delta M_{p,\text{ref}} \approx 100(0.041667)/5.033 = 0.82787 \text{ percentage points}. \tag{E.52}$$

This practical overshoot metric uses the commanded reference. If the terminal display is used as a proxy for the actual settled output, the conventional final-output-normalised overshoot is $(6.229167 - 4.897)/4.897 \times 100 \approx 27.20\%$. Both estimates are below 30%, but the screenshot alone does not establish an exact asymptotic output. Table E.4 compares the analytical, simulated and experimental metrics.

Table E.4: Comparison of calculation, simulation and retained experiment.

Metric	Analytical	Python nominal	Experiment
Reference	5.000 m	5.000 m	5.033 m
Peak	6.25402 m	6.25402 m	≈ 6.229 m
Peak time	22.40597 s	22.405 s	≈ 22.020 s
Reference overshoot	25.0804%	25.0804%	$\approx 23.7665\%$
Tracking accuracy	100% ideal	$\to 100\%$	97.2978% displayed
2% reference settling	64.81 s envelope	54.89 s	Not established

For the 5.033 m reference the 8% band is $5.033(1 \pm 0.08) = 4.63036$–$5.43564$ m. The 2% band is 4.93234–5.13366 m. The terminal height lies inside the first but outside the second. The discrepancy cannot be removed by redefining the settling band around the biased terminal value while claiming reference tracking.

The command-bias calculation uses Equation (2.10): $\delta y = 0.002/(0.01432 \times 0.93) = 0.150177$ m. The illustrative test in Figure F.3 injects a positive 2 mV bias at 70 s; it is a model prediction rather than a recorded disturbance experiment.

Appendix F: Additional Validation Plots

Figure F.1 checks the integrated position response; Figure F.2 compares the three aerodynamic cases listed in Table F.1. The underlying values are retained as CSV and JSON in the `results` folder.

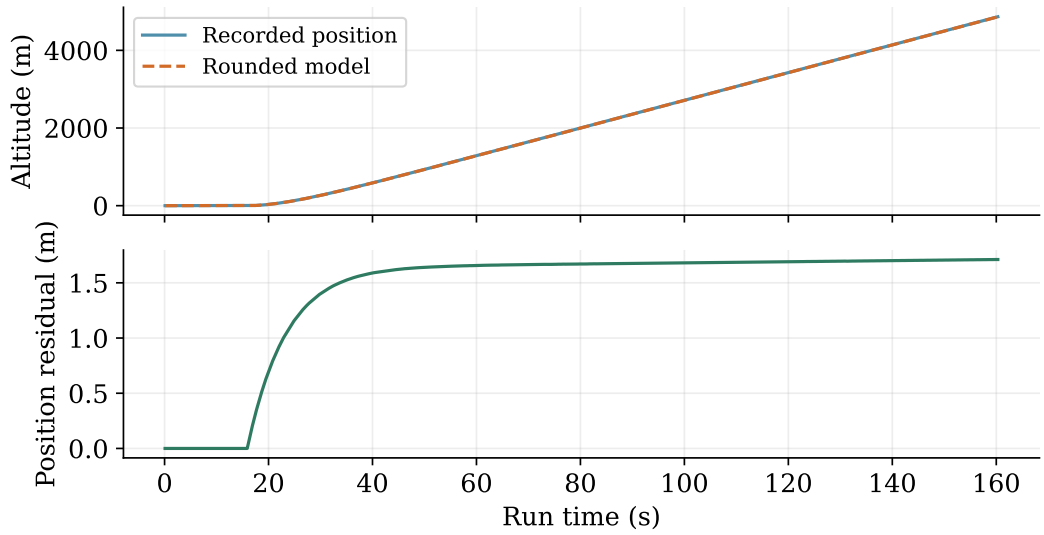


Figure F.1: Position-domain model comparison and residuals for the retained identification run. Code: Python; MATLAB. Data: results.

Table F.1: Predicted performance under aerodynamic uncertainty.

Drag	Peak [m]	Overshoot [%]	$T_{s,2\%}$ [s]
$0.9b_0$	6.473	29.45	70.80
b_0	6.254	25.08	54.89
$1.1b_0$	6.058	21.15	54.59

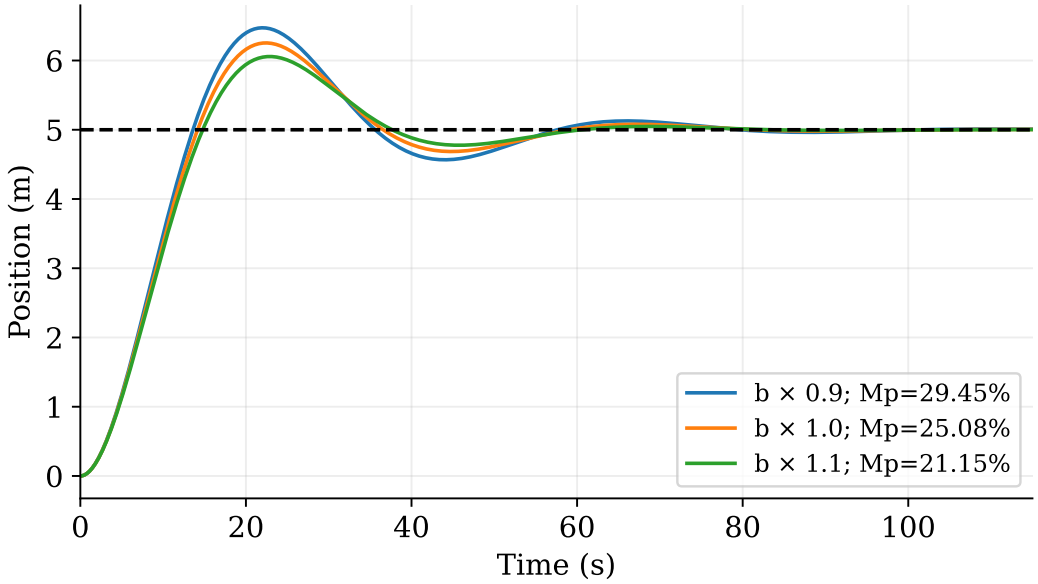


Figure F.2: Predicted altitude responses for $b = 0.9b_0, b_0, 1.1b_0$, with local thrust slope held fixed. Code: Python; MATLAB. Data: results.

Figure F.3 demonstrates sensitivity to a defined input bias. At nominal gain its predicted final displacement is +0.150 m, within the 0.4 m allowance for a 5 m reference.

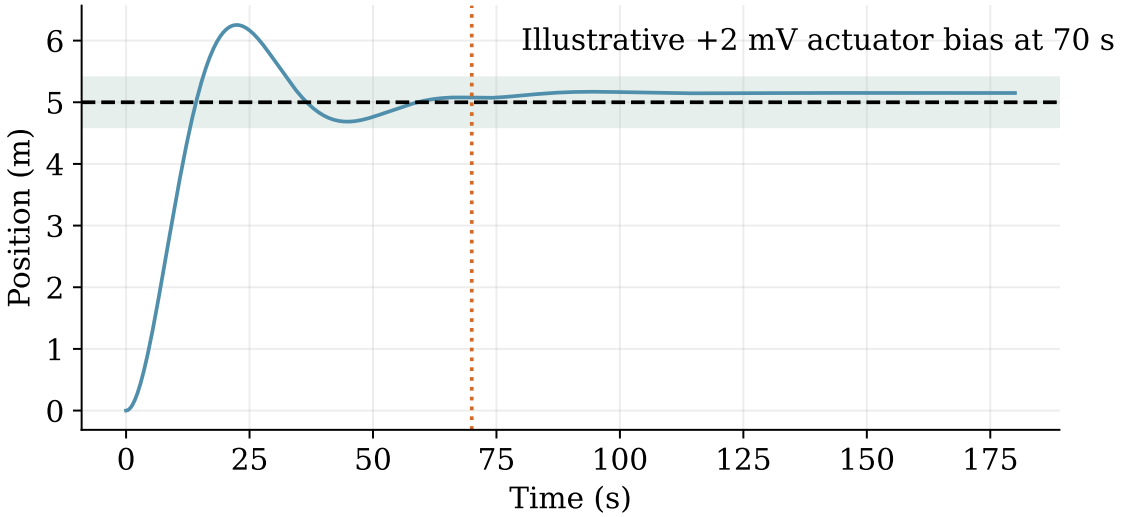


Figure F.3: Illustrative simulated response to +2 mV actuator bias added at 70 s. The shaded region is the 8% tracking band. Code: Python; MATLAB. Data: results.

This simulation tests one stated disturbance amplitude. It does not imply rejection of arbitrary disturbance magnitudes or establish experimental disturbance performance.

Appendix G: Identification Data and Code

G.1 Data Provenance

The unmodified `data/session1/PTRJUN003_raw.csv` contains time, command voltage, sensor voltage, altitude and stored speed. The processed copy excludes 31 pre-reset rows and adds recalculated velocity, midpoint time and model predictions; Table E.2 gives selected calculations.

G.2 Organised Source Code

The numerical pipeline is separated by task. All stages read the shared parameter file; output CSVs and metrics are stored in results. Python/Matplotlib generated every numerical report figure. The MATLAB plotting folder provides counterparts for all numerical figures; its analytical response plots do not require Control System Toolbox. The existing transfer-function checks require that toolbox, and the native model runner requires Simulink.

Table G.1: Code modules and their calculation responsibilities.

Source location	Responsibility
Python modules	Identification, controller response/root locus, robustness, hardware digitisation and disturbance simulation.
MATLAB identification	Backward differences, velocity gain, 63.2% timing and sensor calibration.
MATLAB design	Root-locus geometry, angle/magnitude checks, gain limits and response plots.
MATLAB validation	Independent nominal and aerodynamic controller checks.
Simulink model and tests	Builds the model and logs both reference sequences.
MATLAB plot reproduction	Exports counterparts of all report plots as vector PDFs and PNGs.
Worked calculations	Separate calculation PDF, LaTeX equations and topic index.

Run the Python launcher from the project root:

```
python -m pip install -r requirements.txt
python analysis/analyse.py
```

In MATLAB, set the Current Folder to `matlab`, then enter:

```
run_all
run_simulink_tests(lab_parameters());
root_locus_design
```

Dependencies and step conventions are recorded in the code guide.

Appendix H: Repository and Reproduction

Supporting files are available in the GitHub repository. Folder links open the corresponding folders on the main branch. Open the complete GitHub repository or use the folder links in Table H.1. Figure, table, equation and appendix references remain internal PDF hyperlinks. The repository guide and calculation index explain how to reproduce each result.

Table H.1: Clickable evidence map and corresponding Git folders.

Folder / evidence link	Contents
`data/session1`	Original identification CSV; source description in Appendix G.
`data/session2`	Original experimental display and Simulink screenshots.
`hardware`	Board photograph, LTspice/KiCad design sources and circuit notes.
`analysis`	Python identification, simulation, digitisation and tolerance calculations.
`matlab`	Controller verification and Simulink model builder.
`results`	Processed CSVs, nominal traces, corners and scalar metrics.
`report`	LaTeX source, figures and compiled report.
`scripts`	Build, link configuration and folder indexes.
`config`	One shared parameter file for MATLAB and Python.
`calculations`	Worked calculation PDF and equation source.

H.1 Build the Report

Extract the ZIP, keep its folder structure, install the Python requirements, and run:

```
python -m pip install -r requirements.txt
bash scripts/build.sh
```

The build regenerates numerical results and figures, then compiles `report/main.tex` with `latexmk -pdf`. A LaTeX distribution with TikZ and Circuitikz is required. The package also contains the compiled PDF for immediate viewing.

H.2 GitHub Folder Links

The report uses the repository supplied for this project:

NateDogg247 / EEE3094S—Lab-2-PTRJUN003-

The shared branch setting is `main`. Supporting-folder URLs use `/tree/main/`; individual source-file links use `/blob/main/`. `report/repository.tex` stores these settings centrally. Recompile the report after changing that file.

H.3 Page Count and Reproduction

The main report runs from Introduction on page 1 to Final Remarks on page 14. The title, table of contents, lists of figures and tables, references and appendices are excluded from this count. References use R-prefixed page numbers and appendices use A-prefixed page numbers.

Click any numerical plot to open its Python source folder; each caption also links the MATLAB reproduction folder and numerical results. The numerical pipeline regenerates identification, root-locus, response, tolerance, Simulink-evidence and experimental metrics. The report and calculation supplement retain the corresponding worked equations. The available record contains one identification run and one controller display; it cannot establish repeated-trial averages or an assessor-awarded demonstration mark.